

Course List creation Final 1

<https://gemini.google.com/app/5976085ca058acb1>

User prompt: we have single Framer react component code here. can you please help me to devide it based on it's responsibility? course/ |
 CourseList.tsx ← main/orchestrator | CourseCard.tsx ← single card | useCourses.ts ← API/state | course.types.ts ←
 types | course.utils.ts ← formatting/helpers

Response: Here is the refactored code split into the five designated files based on clean separation of concerns.

course/course.types.ts

Responsibility: Holds all TypeScript interfaces and types for API responses and component props.

```
export interface Course {
  courseName: string
  courseCode: string
  description: string
  mainCategory: string
  shortCourse?: string
  courseType?: string
  pricePaise: number
  priceUsdCents?: number
  mangoId?: string
  refundable: boolean
}

export interface CourseListProps {
  apiUrl: string
}
```

course/course.utils.ts

Responsibility: Pure helper functions, formatting logic, and fallback mock/default constants.

```
import { Course } from "../course.types"

export const DEFAULT_COURSE: Course = {
  courseName: "Instagram Growth Lab",
  courseCode: "instagram-growth-lab",
  description:
    "Master Reels, carousels, and community building with a repeatable content calendar built for creators who w
  mainCategory: "Social Media",
  shortCourse: "Instagram",
  courseType: "Original",
  pricePaise: 149900,
  priceUsdCents: 2999,
  mangoId: "b2c3d4e5f67890123456789a",
  refundable: true,
}

/**
 * Formats price in paise to an INR currency string.
 */
export function formatPricePaiseToINR(pricePaise: number = 0): string {
  return new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR",
    maximumFractionDigits: 0,
  }).format(pricePaise / 100)
}
```

course/useCourses.ts

Responsibility: Encapsulates data fetching, error handling, normalization, and state management.

```
import { useState, useEffect } from "react"
import { Course } from "../course.types"
import { DEFAULT_COURSE } from "../course.utils"

export function useCourses(apiUrl: string) {
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState<boolean>(true)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    console.log("[useCourses] Current apiUrl:", apiUrl)

    if (!apiUrl || apiUrl.trim() === "") {
      console.warn(
```

```

        "[useCourses] No API URL provided. Falling back to default course."
    )
    setCourses([DEFAULT_COURSE])
    setLoading(false)
    return
}

const fetchCourses = async () => {
    setLoading(true)
    setError(null)
    console.log("[useCourses] Initiating fetch to:", apiUrl)

    try {
        const response = await fetch(apiUrl)
        console.log("[useCourses] HTTP Status:", response.status)

        if (!response.ok) {
            throw new Error(`Server returned HTTP ${response.status}`)
        }

        const data = await response.json()
        console.log("[useCourses] Raw API Response Data:", data)

        // Normalize array or object response formats
        if (Array.isArray(data)) {
            setCourses(data)
        } else if (data && typeof data === "object") {
            const list = data.courses || data.data || [data]
            setCourses(Array.isArray(list) ? list : [data])
        } else {
            setCourses([DEFAULT_COURSE])
        }
    } catch (err: any) {
        console.error("[useCourses] Fetch error caught:", err)
        setError(err.message || "Failed to fetch courses.")
    } finally {
        setLoading(false)
    }
}

fetchCourses()
}, [apiUrl])

return { courses, loading, error }
}

```

course/CourseCard.tsx

Responsibility: Presentational component dedicated purely to rendering an individual course item.

```

import * as React from "react"
import { Course } from "../course.types"
import { DEFAULT_COURSE, formatPricePaiseToINR } from "../course.utils"

interface CourseCardProps {
    course?: Course
}

export function CourseCard({ course = DEFAULT_COURSE }: CourseCardProps) {
    const rawPrice = course?.pricePaise ?? 0
    const formattedPrice = formatPricePaiseToINR(rawPrice)

    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed rgba(255, 255, 255, 0.3)",
                borderRadius: "12px",
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "14px",
                color: "#ffffff",
                fontFamily: "Inter, sans-serif",
                width: "300px",
                boxSizing: "border-box",
            }}
        >
            { /* Header: Category & Badge */ }
        </div>
    )
}

```

```

<div
  style={{
    display: "flex",
    justifyContent: "space-between",
    alignItems: "center",
  }}
>
  <span
    style={{
      fontSize: "12px",
      fontWeight: 500,
      textTransform: "uppercase",
      letterSpacing: "0.05em",
      color: "rgba(255, 255, 255, 0.6)",
    }}
  >
    {course?.mainCategory || "General"}
  </span>

  {course?.courseType && (
    <span
      style={{
        fontSize: "11px",
        backgroundColor: "rgba(255, 255, 255, 0.12)",
        padding: "2px 8px",
        borderRadius: "999px",
        color: "#ffffff",
      }}
    >
      {course.courseType}
    </span>
  )}
</div>

{/* Course Title */}
<h3
  style={{
    fontSize: "20px",
    fontWeight: 700,
    margin: 0,
    lineHeight: "1.2",
  }}
>
  {course?.courseName || "Untitled Course"}
</h3>

{/* Description */}
<p
  style={{
    fontSize: "14px",
    lineHeight: "1.5",
    margin: 0,
    color: "rgba(255, 255, 255, 0.75)",
    flexGrow: 1,
  }}
>
  {course?.description || "No description provided."}
</p>

{/* Price */}
<div
  style={{
    fontSize: "22px",
    fontWeight: 700,
    marginTop: "12px",
  }}
>
  {formattedPrice}
</div>

{/* Refundable Badge */}
{course?.refundable && (
  <div
    style={{
      fontSize: "13px",
      display: "flex",
      alignItems: "center",
      gap: "6px",
      color: "rgba(255, 255, 255, 0.9)",
    }}
  >
    <div
      style={{
        width: 10px,
        height: 10px,
        background-color: "#28a745",
        border-radius: 50%,
      }}
    >
    </div>
    {course?.refundable}
  </div>
)}

```

```

    }}
  >
    <span style={{ color: "#4ade80", fontWeight: "bold" }}>
      ✓
    </span>
    <span>Refundable</span>
  </div>
)}
</div>
)
}

```

course/CourseList.tsx

Responsibility: Orchestrator component handling grid layout, UI lifecycle states (loading/error/empty), and Framer property controls.

```

import * as React from "react"
import { addPropertyControls, ControlType } from "framer"
import { CourseCard } from "../CourseCard"
import { useCourses } from "../useCourses"
import { CourseListProps } from "../course.types"

export default function CourseList({ apiUrl }: CourseListProps) {
  const { courses, loading, error } = useCourses(apiUrl)

  if (loading) {
    return (
      <div
        style={{
          color: "rgba(255, 255, 255, 0.7)",
          padding: "24px",
          fontFamily: "Inter, sans-serif",
        }}
      >
        Loading courses from API...
      </div>
    )
  }

  if (error) {
    return (
      <div
        style={{
          color: "#ff6b6b",
          padding: "24px",
          fontFamily: "Inter, sans-serif",
        }}
      >
        <div>Error loading courses: {error}</div>
        <div
          style={{ fontSize: "12px", marginTop: "8px", opacity: 0.8 }}
        >
          Check Developer Tools Console for full details.
        </div>
      </div>
    )
  }

  if (courses.length === 0) {
    return (
      <div
        style={{
          color: "rgba(255, 255, 255, 0.5)",
          padding: "24px",
          fontFamily: "Inter, sans-serif",
        }}
      >
        No courses available.
      </div>
    )
  }

  return (
    <div
      style={{
        display: "flex",
        flexWrap: "wrap",
        gap: "24px",
        justifyContent: "flex-start",

```

```

        padding: "20px",
        width: "100%",
        boxSizing: "border-box",
      }}
    >
    {courses.map((item, index) => (
      <CourseCard
        key={item?.mangoId || item?.courseCode || `course-${index}`}
        course={item}
      />
    ))}
  </div>
)
}

// Framer Defaults & Canvas Controls
CourseList.defaultProps = {
  apiUrl: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
}

addPropertyControls(CourseList, {
  apiUrl: {
    type: ControlType.String,
    title: "API URL",
    defaultValue:
      "https://syncsphere-hiv6.onrender.com/assignment/course-data",
  },
})

```

User prompt: In 'CourseList' Component - I want to show a Circular Loader while fetching data. so can you give that chagne please. No need to write everything, suggest the require change only

Response: Here is the updated loading block for CourseList.tsx using a CSS keyframe circular spinner:

```

if (loading) {
  return (
    <div
      style={{
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        justifyContent: "center",
        gap: "16px",
        padding: "48px 24px",
        width: "100%",
        fontFamily: "Inter, sans-serif",
        color: "rgba(255, 255, 255, 0.7)",
      }}
    >
      <style>{`
        @keyframes spin {
          0% { transform: rotate(0deg); }
          100% { transform: rotate(360deg); }
        }
      `}</style>
      <div
        style={{
          width: "36px",
          height: "36px",
          borderRadius: "50%",
          border: "3px solid rgba(255, 255, 255, 0.15)",
          borderTopColor: "#ffffff",
          animation: "spin 0.8s linear infinite",
        }}
      />
      <span style={{ fontSize: "14px" }}>Loading courses...</span>
    </div>
  )
}

```

User prompt: For error state, it's not align with our theme. think in user's prespective. what is best way to handle the error state? Let's assume that API would return two type of error : - 500 Internal server error { "detail": "FAAAAAAAAAAAAA"} - 404 not found. { "detail": "gg"} we need to handle wisely on UI. Suggest good change.

Response: From a user's perspective, raw error codes like HTTP 500 or raw backend dumps (FAAAAAAAAAAAAA) confuse users, look broken, and leave them with no next step.

A good error state should:

1. **Explain the issue in plain English** (distinguishing server issues from missing resources).
2. **Blend with the page aesthetic** (clean typography, subtle borders, no jarring raw red text).
3. **Offer an immediate recovery action** (e.g., a **"Try Again"** button to retry the fetch without refreshing the entire page).

1. Update useCourses.ts (Capture status & expose refetch)

Update your hook so it tracks the HTTP status code and exposes a refetch function:

```
import { useState, useEffect, useCallback } from "react"
import { Course } from "../course.types"
import { DEFAULT_COURSE } from "../course.utils"

export interface FetchError {
  status?: number
  title: string
  message: string
}

export function useCourses(apiUrl: string) {
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState<boolean>(true)
  const [error, setError] = useState<FetchError | null>(null)

  const fetchCourses = useCallback(async () => {
    if (!apiUrl || apiUrl.trim() === "") {
      setCourses([DEFAULT_COURSE])
      setLoading(false)
      return
    }

    setLoading(true)
    setError(null)

    try {
      const response = await fetch(apiUrl)

      if (!response.ok) {
        if (response.status === 404) {
          setError({
            status: 404,
            title: "Courses Not Found",
            message: "We couldn't find the course catalog you're looking for.",
          })
        } else if (response.status >= 500) {
          setError({
            status: 500,
            title: "Temporarily Unavailable",
            message: "We're experiencing issues on our end. Please give it another try in a moment.",
          })
        } else {
          setError({
            status: response.status,
            title: "Unable to Load Courses",
            message: "An unexpected error occurred while loading content.",
          })
        }
      }
      return

      const data = await response.json()
      if (Array.isArray(data)) {
        setCourses(data)
      } else if (data && typeof data === "object") {
        const list = data.courses || data.data || [data]
        setCourses(Array.isArray(list) ? list : [data])
      } else {
        setCourses([DEFAULT_COURSE])
      }
    } catch {
      setError({
        title: "Connection Error",
        message: "Unable to reach the server. Please check your internet connection.",
      })
    } finally {
      setLoading(false)
    }
  }, [apiUrl])
}
```

```
    }, [apiUrl])

    useEffect(() => {
      fetchCourses()
    }, [fetchCourses])

    return { courses, loading, error, refetch: fetchCourses }
  }
}
```

2. Update CourseList.tsx (Add the Styled Error UI)

Replace the old `if (error)` block with a clean, branded card featuring an icon and a **"Try Again"** button:

```
export default function CourseList({ apiUrl }: CourseListProps) {
  const { courses, loading, error, refetch } = useCourses(apiUrl)

  // ... (keep loading state here)

  if (error) {
    return (
      <div
        style={{
          display: "flex",
          flexDirection: "column",
          alignItems: "center",
          justifyContent: "center",
          textAlign: "center",
          padding: "48px 24px",
          margin: "20px auto",
          maxWidth: "420px",
          borderRadius: "16px",
          backgroundColor: "rgba(0, 0, 0, 0.03)",
          border: "1px solid rgba(0, 0, 0, 0.08)",
          fontFamily: "Inter, sans-serif",
          color: "#111827",
        }}
      >
        {/* Icon Badge */}
        <div
          style={{
            width: "48px",
            height: "48px",
            borderRadius: "50%",
            backgroundColor: error.status === 404 ? "#f3f4f6" : "#fee2e2",
            display: "flex",
            alignItems: "center",
            justifyContent: "center",
            fontSize: "20px",
            marginBottom: "16px",
          }}
        >
          {error.status === 404 ? "🔍" : "⚠️"}
        </div>

        {/* Error Title */}
        <h3
          style={{
            fontSize: "18px",
            fontWeight: 600,
            margin: "0 0 8px 0",
            color: "#111827",
          }}
        >
          {error.title}
        </h3>

        {/* Friendly Description */}
        <p
          style={{
            fontSize: "14px",
            lineHeight: 1.5,
            color: "#6b7280",
            margin: "0 0 20px 0",
          }}
        >
          {error.message}
        </p>

        {/* Recovery Action */}

```

```

    {error.status !== 404 && (
      <button
        onClick={() => refetch()}
        style={{
          padding: "9px 20px",
          backgroundColor: "#111827",
          color: "#ffffff",
          fontSize: "13px",
          fontWeight: 500,
          borderRadius: "8px",
          border: "none",
          cursor: "pointer",
          transition: "opacity 0.2s",
        }}
        onMouseEnter={(e) => (e.currentTarget.style.opacity = "0.85")}
        onMouseLeave={(e) => (e.currentTarget.style.opacity = "1")}
      >
        Try Again
      </button>
    )}
  </div>
)
}

// ... (render CourseCard items)
}
```

Visual & UX Comparison

Old State	New State
Red monospace error text floating on canvas	Subtle rounded neutral card aligned with the light layout
Shows raw technical error: HTTP 500	Shows clear message: "Temporarily Unavailable / Please give it another try"
Mentions "Check DevTools Console"	Hides technical jargon from the end user
Requires full browser refresh	Provides an interactive "Try Again" button

User prompt: For your information - I have removed our hardcoded Default Course entirely. Now let's discuss new Requirement on top of our current work - Current Price visuals : - we are using 'Intl.NumberFormat' Function which consider 'en-IN' country and convert the Paisa into Ruppee. which is good. Now we have an API, which Returns us Three type of response : "country_code": "IN" , "country_code": "US" , and might be 404 with response "detail": "maybe turn it on and off?" based on that API's response we have to decide which currency we are going to show by converting in appropriate country. against we must be keep calculation from Paisa to Ruppee & show dollars from priceUsdCents (Dividing by 100) we should also handle the case when country API failed but Course API works : Here is my idea to handle - Show the courses normally Use a clear fallback currency, preferably USD Show a small message like "Unable to detect your region. Showing prices in USD." Provide Retry for the country request. and we don't really need the Country result because there are no courses to price in case of 'Course API fail' let's go with code changes for this now. Follow best practise for code.

Response: Here is the clean, modular implementation designed to handle the dynamic Geo/Country API, multi-currency conversion, fallback to USD, and resilient non-blocking UI notifications.

1. course/course.types.ts

Added CountryCode, API response shapes, and updated component props.

```

export interface Course {
  courseName: string
  courseCode: string
  description: string
  mainCategory: string
  shortCourse?: string
  courseType?: string
  pricePaise: number
  priceUsdCents?: number
  mangoId?: string
  refundable: boolean
}

export type CountryCode = "IN" | "US" | string

export interface CountryApiResponse {
  country_code?: CountryCode
  detail?: string
}

export interface FetchError {
```



```

    status?: number
    title: string
    message: string
  }

export interface CourseListProps {
  apiUrl: string
  countryApiUrl: string
}

```

2. course/course.utils.ts

Centralizes currency conversion and price formatting. Defaults gracefully to USD.

```

import { Course, CountryCode } from "../course.types"

/**
 * Formats course price based on detected country.
 * - "IN" -> INR (from pricePaise)
 * - "US" / fallback -> USD (from priceUsdCents)
 */
export function formatCoursePrice(course: Course, countryCode?: CountryCode): string {
  const isIndia = countryCode?.toUpperCase() === "IN"

  if (isIndia) {
    const inrAmount = (course.pricePaise ?? 0) / 100
    return new Intl.NumberFormat("en-IN", {
      style: "currency",
      currency: "INR",
      maximumFractionDigits: 0,
    }).format(inrAmount)
  }

  // Default Fallback: USD
  const usdAmount = (course.priceUsdCents ?? 0) / 100
  return new Intl.NumberFormat("en-US", {
    style: "currency",
    currency: "USD",
    maximumFractionDigits: 0,
  }).format(usdAmount)
}

```

3. course/useCountry.ts (New Hook)

Isolates country detection, error tracking, and retry logic without blocking course rendering.

```

import { useState, useEffect, useCallback } from "react"
import { CountryCode, CountryApiResponse } from "../course.types"

export function useCountry(countryApiUrl: string) {
  const [countryCode, setCountryCode] = useState<CountryCode>("US")
  const [countryLoading, setCountryLoading] = useState<boolean>(true)
  const [countryError, setCountryError] = useState<boolean>(false)

  const fetchCountry = useCallback(async () => {
    if (!countryApiUrl || countryApiUrl.trim() === "") {
      setCountryCode("US")
      setCountryLoading(false)
      return
    }

    setCountryLoading(true)
    setCountryError(false)

    try {
      const response = await fetch(countryApiUrl)

      if (!response.ok) {
        throw new Error(`Country API status: ${response.status}`)
      }

      const data: CountryApiResponse = await response.json()

      if (data?.country_code) {
        setCountryCode(data.country_code)
      } else {
        // Fallback to USD if no valid code in body
        setCountryCode("US")
        setCountryError(true)
      }
    }
  }, [countryApiUrl])
}

```

```

    }
  } catch (err) {
    console.warn("[useCountry] Detection failed, falling back to USD:", err)
    setCountryCode("US")
    setCountryError(true)
  } finally {
    setCountryLoading(false)
  }
}, [countryApiUrl])

useEffect(() => {
  fetchCountry()
}, [fetchCountry])

return {
  countryCode,
  countryLoading,
  countryError,
  retryCountry: fetchCountry,
}
}
}

```

4. course/useCourses.ts

Updated to remove hardcoded fallbacks and return standard error structures.

```

import { useState, useEffect, useCallback } from "react"
import { Course, FetchError } from "../course.types"

export function useCourses(apiUrl: string) {
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState<boolean>(true)
  const [error, setError] = useState<FetchError | null>(null)

  const fetchCourses = useCallback(async () => {
    if (!apiUrl || apiUrl.trim() === "") {
      setCourses([])
      setLoading(false)
      return
    }

    setLoading(true)
    setError(null)

    try {
      const response = await fetch(apiUrl)

      if (!response.ok) {
        if (response.status === 404) {
          setError({
            status: 404,
            title: "Courses Not Found",
            message: "We couldn't find the courses you are looking for.",
          })
        } else if (response.status >= 500) {
          setError({
            status: 500,
            title: "Server Error",
            message: "Our servers are experiencing an issue. Please try again shortly.",
          })
        } else {
          setError({
            status: response.status,
            title: "Unable to Load Courses",
            message: "An unexpected error occurred while fetching course data.",
          })
        }
      }
      return
    }

    const data = await response.json()
    if (Array.isArray(data)) {
      setCourses(data)
    } else if (data && typeof data === "object") {
      const list = data.courses || data.data || [data]
      setCourses(Array.isArray(list) ? list : [])
    } else {
      setCourses([])
    }
  })
}

```

```

    } catch {
      setError({
        title: "Network Error",
        message: "Unable to reach the server. Please check your internet connection.",
      })
    } finally {
      setLoading(false)
    }
  }, [apiUrl])

  useEffect(() => {
    fetchCourses()
  }, [fetchCourses])

  return { courses, loading, error, refetch: fetchCourses }
}

```

5. course/CourseCard.tsx

Accepts the active countryCode and formats the price accordingly.

```

import * as React from "react"
import { Course, CountryCode } from "../course.types"
import { formatCoursePrice } from "../course.utils"

interface CourseCardProps {
  course: Course
  countryCode?: CountryCode
}

export function CourseCard({ course, countryCode }: CourseCardProps) {
  const formattedPrice = formatCoursePrice(course, countryCode)

  return (
    <div
      style={{
        backgroundColor: "#1a1a1a",
        border: "1px dashed rgba(255, 255, 255, 0.3)",
        borderRadius: "12px",
        padding: "24px",
        display: "flex",
        flexDirection: "column",
        gap: "14px",
        color: "#ffffff",
        fontFamily: "Inter, sans-serif",
        width: "300px",
        boxSizing: "border-box",
      }}
    >
      { /* Header: Category & Badge */ }
      <div
        style={{
          display: "flex",
          justifyContent: "space-between",
          alignItems: "center",
        }}
      >
        <span
          style={{
            fontSize: "12px",
            fontWeight: 500,
            textTransform: "uppercase",
            letterSpacing: "0.05em",
            color: "rgba(255, 255, 255, 0.6)",
          }}
        >
          {course.mainCategory || "General"}
        </span>

        {course.courseType && (
          <span
            style={{
              fontSize: "11px",
              backgroundColor: "rgba(255, 255, 255, 0.12)",
              padding: "2px 8px",
              borderRadius: "999px",
              color: "#ffffff",
            }}
          >

```

```

        {course.courseType}
      </span>
    )}
  </div>

  { /* Title */ }
  <h3
    style={{
      fontSize: "20px",
      fontWeight: 700,
      margin: 0,
      lineHeight: "1.2",
    }}
  >
    {course.courseName || "Untitled Course"}
  </h3>

  { /* Description */ }
  <p
    style={{
      fontSize: "14px",
      lineHeight: "1.5",
      margin: 0,
      color: "rgba(255, 255, 255, 0.75)",
      flexGrow: 1,
    }}
  >
    {course.description || "No description provided."}
  </p>

  { /* Dynamic Localized Price */ }
  <div
    style={{
      fontSize: "22px",
      fontWeight: 700,
      marginTop: "12px",
    }}
  >
    {formattedPrice}
  </div>

  { /* Refundable Tag */ }
  {course.refundable && (
    <div
      style={{
        fontSize: "13px",
        display: "flex",
        alignItems: "center",
        gap: "6px",
        color: "rgba(255, 255, 255, 0.9)",
      }}
    >
      <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
      <span>Refundable</span>
    </div>
  )}
</div>
)
}

```

6. course/CourseList.tsx

Orchestrates both hooks. If the country API fails, it displays a non-intrusive alert banner with a **Retry** button above the courses while defaulting prices to USD.

```

import * as React from "react"
import { addPropertyControls, ControlType } from "framer"
import { CourseCard } from "../CourseCard"
import { useCourses } from "../useCourses"
import { useCountry } from "../useCountry"
import { CourseListProps } from "../course.types"

export default function CourseList({ apiUrl, countryApiUrl }: CourseListProps) {
  const { courses, loading, error, refetch } = useCourses(apiUrl)
  const { countryCode, countryError, retryCountry } = useCountry(countryApiUrl)

  // 1. Loading State (Spinner)
  if (loading) {
    return (

```

```

<div
  style={{
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    justifyContent: "center",
    gap: "16px",
    padding: "48px 24px",
    width: "100%",
    fontFamily: "Inter, sans-serif",
    color: "rgba(0, 0, 0, 0.6)",
  }}
>
  <style>`
    @keyframes spin {
      0% { transform: rotate(0deg); }
      100% { transform: rotate(360deg); }
    }
  `</style>
  <div
    style={{
      width: "36px",
      height: "36px",
      borderRadius: "50%",
      border: "3px solid rgba(0, 0, 0, 0.1)",
      borderTopColor: "#111827",
      animation: "spin 0.8s linear infinite",
    }}
  />
  <span style={{ fontSize: "14px" }}>Loading courses...</span>
</div>
)
}

// 2. Fatal Course API Error State
if (error) {
  return (
    <div
      style={{
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        justifyContent: "center",
        textAlign: "center",
        padding: "48px 24px",
        margin: "20px auto",
        maxWidth: "420px",
        borderRadius: "16px",
        backgroundColor: "rgba(0, 0, 0, 0.03)",
        border: "1px solid rgba(0, 0, 0, 0.08)",
        fontFamily: "Inter, sans-serif",
        color: "#111827",
      }}
    >
      <div
        style={{
          width: "48px",
          height: "48px",
          borderRadius: "50%",
          backgroundColor: error.status === 404 ? "#f3f4f6" : "#fee2e2",
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          fontSize: "20px",
          marginBottom: "16px",
        }}
      >
        {error.status === 404 ? "🔍" : "⚠️"}
      </div>
      <h3 style={{ fontSize: "18px", fontWeight: 600, margin: "0 0 8px 0" }}>
        {error.title}
      </h3>
      <p style={{ fontSize: "14px", lineHeight: 1.5, color: "#6b7280", margin: "0 0 20px 0" }}>
        {error.message}
      </p>
      {error.status !== 404 && (
        <button
          onClick={() => refetch()}
          style={{

```

```

        padding: "9px 20px",
        backgroundColor: "#111827",
        color: "#ffffff",
        fontSize: "13px",
        fontWeight: 500,
        borderRadius: "8px",
        border: "none",
        cursor: "pointer",
      }}
    >
      Try Again
    </button>
  )}
</div>
)
}

// 3. Empty State
if (courses.length === 0) {
  return (
    <div
      style={{
        color: "rgba(0, 0, 0, 0.5)",
        padding: "32px",
        textAlign: "center",
        fontFamily: "Inter, sans-serif",
      }}
    >
      No courses available at this time.
    </div>
  )
}

// 4. Success State (Courses loaded)
return (
  <div style={{ display: "flex", flexDirection: "column", width: "100%", gap: "16px" }}>
    { /* Non-blocking Region Error Banner */ }
    {countryError && (
      <div
        style={{
          margin: "0 20px",
          padding: "10px 16px",
          backgroundColor: "#ffffeb",
          border: "1px solid #fef3c7",
          borderRadius: "8px",
          display: "flex",
          alignItems: "center",
          justifyContent: "space-between",
          fontSize: "13px",
          color: "#92400e",
          fontFamily: "Inter, sans-serif",
        }}
      >
        <span>⚠️ Unable to detect your region. Showing prices in USD.</span>
        <button
          onClick={() => retryCountry()}
          style={{
            background: "transparent",
            border: "none",
            color: "#b45309",
            fontWeight: 600,
            cursor: "pointer",
            textDecoration: "underline",
            padding: "0 4px",
            fontSize: "13px",
          }}
        >
          Retry
        </button>
      </div>
    )}

    { /* Course Grid */ }
    <div
      style={{
        display: "flex",
        flexWrap: "wrap",
        gap: "24px",
        justifyContent: "flex-start",

```

```

        padding: "20px",
        width: "100%",
        boxSizing: "border-box",
      }}
    >
    {courses.map((item, index) => (
      <CourseCard
        key={item?.mangoId || item?.courseCode || `course-${index}`}
        course={item}
        countryCode={countryCode}
      />
    ))}
  </div>
</div>
)
}

// Framer Defaults & Canvas Controls
CourseList.defaultProps = {
  apiUrl: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
  countryApiUrl: "https://syncsphere-hiv6.onrender.com/assignment/country-data",
}

addPropertyControls(CourseList, {
  apiUrl: {
    type: ControlType.String,
    title: "Course API URL",
    defaultValue: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
  },
  countryApiUrl: {
    type: ControlType.String,
    title: "Country API URL",
    defaultValue: "https://syncsphere-hiv6.onrender.com/assignment/country-data",
  },
})

```

User prompt: Let's Move for next Requirement of UI (don't write every code again, just give me the necessary change) Refer attached image of current UI - New points : we only shows discription upto 2 lines only. instead of cutting the string by character we will go with approach 'CSS line clamp → ellipsis' I'd make sure the description area has a fixed/minimum height equivalent to 2 lines, so a short description doesn't cause the price to jump upward compared with other cards. can you help me with code please?

Response: Here is the updated `<p>` description block for `CourseCard.tsx`.

Replace the existing `<p>` element with this:

```

{ /* Description (2-line clamp with fixed height) */ }
<p
  style={{
    fontSize: "14px",
    lineHeight: "1.5",
    height: "42px", /* 14px * 1.5 * 2 lines = 42px */
    display: "-webkit-box",
    WebkitLineClamp: 2,
    WebkitBoxOrient: "vertical",
    overflow: "hidden",
    textOverflow: "ellipsis",
    margin: 0,
    color: "rgba(255, 255, 255, 0.75)",
  }}
>
  {course?.description || "No description provided."}
</p>

```

Why this works:

- `-webkit-line-clamp: 2`: Truncates text at exactly 2 lines and adds ... at the end.
- `height: 42px`: Locks the height to precisely 2 lines ($14\text{px} \times 1.5 \text{ line-height} \times 2 = 42\text{px}$), keeping the price tag aligned across all cards even if a description is only 1 line long.

User prompt: Here is the current UI of grid. it's build with framer and react code we have written. Now let me comes on the point - The grid is not centrally aligned. we have to make it proper distributed in screen width with space and alignment. before we jump on code - I have one doubt : see, in traditional way of development (& even in React code withing Framer) we define the styling and sizing , alignment , position in code itself with CSS and classes. but when same react code bind with Framer's Canvas , it is also provide us to mangae those parameter. so How both works togher. Let's say I want to manage and handle those things for my react component but still need to make sure that it works well , adjust with other Framer component. so how industry handle this? how it works behind the scene? tell me in short - simple and on-point.

Response: Framer treats custom React components as children inside an outer wrapper frame.

The Division of Responsibility

- **Framer Canvas handles the Outer Shell:** Page margins, max-width constraints (e.g., 1200px), breakpoints (Desktop/Tablet/Mobile), and vertical positioning on the page.
- **React Code handles the Inner Engine:** The card rendering, data fetching, internal card spacing, and how items wrap inside the container.

How the Industry Connects Them

1. Make Code 100% Fluid Never hardcode fixed pixel widths on your top-level React container. Write code that occupies width: 100% and adapts to whatever space Framer gives it.

2. Use Modern CSS Grid Inside Code Instead of static width: 300px per card, use responsive CSS Grid:

```
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 24px;
width: 100%;
```

- **Result:** Cards stretch evenly to fill the row and center themselves naturally, leaving no awkward empty gaps on the right.

3. Configure the Canvas Frame On the Framer Canvas properties panel:

- Set Component Width to **Fill** (or give it a **Max Width** like 1200px).
- Set Alignment/Distribution to **Center**.

Summary Matrix

Property	Managed in Canvas	Managed in React Code
Max Page Width & Margins	Yes (Layout / Section controls)	No
Breakpoints / Screen Sizes	Yes (Desktop/Tablet/Mobile variants)	No
Grid Column Auto-Fill & Gaps	Optional (via Property Controls)	Yes (CSS Grid / Flex)
Internal Card Styles & Content	No	Yes (JSX & Props)

User prompt: Great! so here what I understand : Framer manage outer Frame and React responsible for Inner Framer of it's component. ideally we should always provide a flexible width and height for outermost react Frame so that it can manage by Framer properly. correct? so in our case - which code we needs to change? and which changes I need to manully perform in Framer's UI itself. Guide me please.

Response: Your understanding is spot on—keeping the React root at width: 100% lets Framer dictate the boundaries while CSS Grid handles the internal responsiveness.

1. Code Changes

In `CourseCard.tsx`, remove width: "300px" so the card fluidly fills its grid cell:

```
// Inside CourseCard.tsx root div style:
style={{
  backgroundColor: "#1a1a1a",
  border: "1px dashed rgba(255, 255, 255, 0.3)",
  borderRadius: "12px",
  padding: "24px",
  display: "flex",
  flexDirection: "column",
  gap: "14px",
  color: "#ffffff",
  fontFamily: "Inter, sans-serif",
  width: "100%", // <-- Changed from "300px" to "100%"
  boxSizing: "border-box",
}}
```

In `CourseList.tsx`, replace the flex container with auto-fitting CSS Grid:

```
// Inside CourseList.tsx (grid container):
<div
  style={{
    display: "grid",
    gridTemplateColumns: "repeat(auto-fill, minmax(280px, 1fr))",
    gap: "24px",
    width: "100%",
    boxSizing: "border-box",
  }}
>
```



```

{courses.map((item, index) => (
  <CourseCard
    key={item?.mongoId || item?.courseCode || `course-${index}`}
    course={item}
    countryCode={countryCode}
  />
))}
</div>

```

2. Framer Canvas UI Changes

Select your layers on the Framer canvas and set these properties in the right sidebar:

- **Select the CourseList Component Layer:**
 - **Width:** Set to Fill (100%) (not *Fixed*).
 - **Height:** Set to Fit Content.
- **Select the Parent Section / Stack (the outer frame containing your heading & component):**
 - **Width:** Set to Fill (100%).
 - **Max Width:** Set to 1200px (or 1280px based on your site design).
 - **Alignment:** Set horizontal alignment to Center.
 - **Padding:** Add horizontal padding (e.g., 24px or 40px) to prevent cards from touching the screen edges on mobile/smaller screens.

User prompt: Here I attached two Files : 1. Course Card - Card Design Code 2. Theme - Theme variable define. I just want to veritcally align the Price and 'Refundable' label. Vertically align but one at most left and one at most right side of card. It's good if we can make it visual with good Light green badge. (Find a good Color shade align with theme.) - Only write the code which needs to update, instead of writing everything again.

Response: Replace the existing Price and Refundable blocks in `CourseCard.tsx` with this single bottom container:

```

{/* Footer: Price & Refundable Badge */}
<div
  style={{
    display: "flex",
    justifyContent: "space-between",
    alignItems: "center",
    marginTop: `${theme.spacing.sm + 4}px`,
  }}
>
  {/* Left: Dynamic Price */}
  <div
    className="course-card-price"
    style={{
      fontSize: theme.typography.price,
      fontWeight: 700,
      color: theme.colors.text,
      margin: 0,
    }}
  >
    {formattedPrice}
  </div>

  {/* Right: Light Green Badge */}
  {course.refundable && (
    <div
      style={{
        display: "inline-flex",
        alignItems: "center",
        gap: "4px",
        padding: "4px 9px",
        borderRadius: theme.radius.pill,
        backgroundColor: "rgba(34, 197, 94, 0.12)",
        border: "1px solid rgba(34, 197, 94, 0.25)",
        color: "#15803d",
        fontSize: "12px",
        fontWeight: 600,
        letterSpacing: "0.01em",
      }}
    >
      <span style={{ fontSize: "11px", fontWeight: "bold" }}></span>
      <span>Refundable</span>
    </div>
  )}

```

```
    })
  </div>
```

User prompt: can you please Define color in Theme.ts and then let use in Code directly. don't write again, just give me line to define the color

Response: In theme.ts (under colors):

```
// Add inside theme.colors:
badgeSuccessBg: "rgba(34, 197, 94, 0.12)",
badgeSuccessBorder: "rgba(34, 197, 94, 0.25)",
badgeSuccessText: "#15803d",
```

In CourseCard.tsx (updated badge inline styles):

```
backgroundColor: theme.colors.badgeSuccessBg,
border: `1px solid ${theme.colors.badgeSuccessBorder}`,
color: theme.colors.badgeSuccessText,
```

User prompt: here is the code of our Course list which allow to appear the GRID of course cards. Now I want to add Two property control which any one can edit from Framer's UI as well. 1. Add new Title (Left align) 'Explore our courses' before Grid and let's allow anyone can edit it from Framer without touching code. 2. also let's allow to adjust the gap between card to change from Framer as well. Suggest me code change require for this. [do not need to write whole code again, give me neccssary change only]

Response: 1. Update course_types.ts

Add the two optional props to CourseListProps:

```
export interface CourseListProps {
  apiUrl: string
  countryApiUrl: string
  title?: string
  gridGap?: number
}
```

2. Update CourseList.tsx Component Signature

Add title and gridGap with sensible defaults in CourseList.tsx:

```
export default function CourseList({
  apiUrl,
  countryApiUrl,
  title = "Explore our courses",
  gridGap = 24,
}: CourseListProps) {
```

3. Update JSX in CourseList.tsx (Render Title & Dynamic Gap)

In the success return block, insert the title and pass gridGap to the grid container:

```
<div className="course-list-shell">
  /* 1. Section Title */
  {title && (
    <h2
      style={{
        fontSize: "24px",
        fontWeight: 700,
        color: theme.colors.text,
        margin: `0 0 ${theme.spacing.lg}px 0`,
        textAlign: "left",
        fontFamily: theme.typography.fontFamily,
      }}
    >
      {title}
    </h2>
  )}

  /* 2. Grid with Dynamic Gap */
  <div className="course-grid" style={{ gap: `${gridGap}px` }}>
    {courses.map((item, index) => (
      <CourseCard
        key={item?.mangoId ?? item?.courseCode ?? `course-${index}`}
        course={item}
        countryCode={countryCode}
      />
    ))}
  </div>
</div>
```

4. Update defaultProps and addPropertyControls in CourseList.tsx

Register the controls so they appear in Framer's sidebar panel:

```
CourseList.defaultProps = {
  apiUrl: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
  countryApiUrl: "https://syncsphere-hiv6.onrender.com/assignment/country-code",
  title: "Explore our courses",
  gridGap: 24,
}

addPropertyControls(CourseList, {
  title: {
    type: ControlType.String,
    title: "Title",
    defaultValue: "Explore our courses",
  },
  gridGap: {
    type: ControlType.Number,
    title: "Card Gap",
    min: 8,
    max: 64,
    step: 4,
    unit: "px",
    defaultValue: 24,
  },
  apiUrl: {
    type: ControlType.String,
    title: "Course API URL",
    defaultValue: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
  },
  countryApiUrl: {
    type: ControlType.String,
    title: "Country API URL",
    defaultValue: "https://syncsphere-hiv6.onrender.com/assignment/country-code",
  },
})
```

User prompt: Great! it works. not simplify How this Property control works and which thing we need to pass? tell me about the parameter you have set for addPropertycontrols. in short and on-point

Response: addPropertyControls is Framer's bridge that maps React props to visual controls in the Framer design sidebar, allowing non-developers to edit values visually.

How It Works

You pass two arguments to the function:

1. **Component:** The React component target (e.g., CourseList).
2. **Schema Object:** A key-value map where each key matches a component prop name, and the value defines its UI control type and constraints.

Parameters Used in the Setup

- **type** (ControlType.String / ControlType.Number): Defines the UI input type (text box, number slider, toggle, color picker, etc.).
- **title:** The visual label shown next to the input in the Framer properties sidebar.
- **defaultValue:** The initial fallback value used when the component is placed on the canvas.
- **min & max:** Sets the slider bounds for numeric inputs (e.g., limits gap between 8px and 64px).
- **step:** The increment amount when scrubbing or clicking arrow controls (e.g., jumps by 4px).
- **unit:** The visual measurement unit tag displayed inside the input field (e.g., "px", "%", "deg").